

The New Shortest Path Algorithm

Adham Abotarboush
Department of Computer Science
The American University in Cairo
Cairo, Egypt

Ahmed M. Yamany
Department of Computer Science
The American University in Cairo
Cairo, Egypt

Tarek Kamel
Department of Computer Science
The American University in Cairo
Cairo, Egypt

Omar Marey
Department of Computer Science
The American University in Cairo
Cairo, Egypt

Abstract—Left blank intentionally

I. INTRODUCTION

Left blank intentionally

II. LITERATURE SURVEY

The shortest path problem has been a central topic in graph theory and algorithm design for decades. This section reviews four key works that form the historical and theoretical foundation for modern Single-Source Shortest Path (SSSP) algorithms, including the recent “Breaking the Sorting Barrier” (BSB) algorithm explored in our experiment.

A. Classical Foundations: Bellman (1958)

One of the earliest formal treatments of shortest path computation was introduced by Bellman in 1958 through a dynamic programming formulation. Bellman presented a recurrence relation that computes the shortest path from any node to a destination by repeatedly relaxing all edges in the graph. This method, later known as the Bellman–Ford algorithm, guarantees correctness even in the presence of negative edge weights. However, due to its repeated relaxation over all edges, the approach runs in $O(V \cdot E)$ or $O(n \cdot m)$ time, which makes it impractical for large graphs. This method laid the groundwork by framing shortest paths as a dynamic programming problem, but its high computational cost motivated the search for faster solutions.

B. Greedy Optimization: Dijkstra (1959)

Dijkstra’s 1959 algorithm provided a major breakthrough in efficient shortest-path computation for graphs with non-negative weights. Instead of relaxing all edges repeatedly, Dijkstra introduced a greedy strategy that always expands the node with the smallest known tentative distance. Using a basic array-based priority structure, the original algorithm achieved $O(n^2)$ time. Using a min-priority queue (heap) to select the next vertex, one can achieve a runtime of $O(m + n \log n)$ reflecting the cost of heap operations for each of the n vertices. Dijkstra’s algorithm remains the backbone of many real-world applications, including network routing and navigation systems. The limitation, however, is that Dijkstra’s algorithm

assumes no negative weights (it will fail or give incorrect results if negative edge weights are present). Additionally, the need for a priority queue that maintains a global ordering of distances, effectively imposing a “sorting barrier” that algorithms relying on comparisons cannot avoid.

C. Graph Algorithms in Practice: Aggarwal (2016)

Aggarwal’s 2016 survey on graph algorithms places classical shortest-path methods in the context of modern, large-scale real-world networks. The survey highlights the widespread use of graph algorithms in social networks, transportation systems, communication networks, and biological systems. It also emphasizes that although Dijkstra and Bellman–Ford remain essential, the explosive growth of graph sizes in practice has exposed their performance limitations. As graphs reach millions of nodes and edges, even $O(m + n \log n)$ approaches can become costly, motivating both improvements and entirely new algorithmic frameworks. Aggarwal notes that various strategies have been developed to cope with scale, such as preprocessing techniques, hierarchical road-network heuristics (like goal-directed search and landmark-based “A*” speedups for maps), and parallel/distributed algorithms. Therefore, his work provided a high-level justification for research into faster SSSP algorithms, including those that reduce or avoid sorting overhead.

D. Modern Breakthrough: Breaking the Sorting Barrier SSSP (2025)

The 2025 paper “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths” introduces the first deterministic SSSP algorithm that asymptotically outperforms Dijkstra’s $O(m + n \log n)$ complexity for sparse directed graphs with non-negative weights. The key idea is to reduce the dependence on strict priority queue ordering by blending limited Bellman–Ford–style relaxations with Dijkstra-like frontier expansion. By structuring relaxations in recursive layers and avoiding full-distance sorting, the algorithm achieves an improved time complexity of $O(m \cdot \log^{2/3} n)$. The result is a provably faster algorithm that “breaks the sorting barrier,” meaning it no longer spends $(n \log n)$ time sorting vertices by distance. Although theoretically significant, the algorithm is

complex to implement directly, inspiring practical adaptations such as the Radix Heap used in our experiment.

E. Connection to the Experiment

The progression from Bellman’s and Dijkstra’s classical algorithms to the 2025 SSSP breakthrough directly informed our experimental work. The literature showed that Dijkstra’s algorithm (using a binary heap) has a fundamental sorting overhead, whereas newer approaches aim to reduce that overhead. To explore this in practice, we implemented a radix heap version of the shortest path algorithm inspired by the SSSP paper’s ideas. A radix heap is a bucket-based priority queue that avoids general-purpose sorting by leveraging the binary representation of distance values. This data structure was suggested by modern research as a way to achieve better performance for graphs with integer weights. In our experiment, we compared classic Dijkstra (binary heap) versus a Radix Heap-based SSSP implementation. This directly tests the claims from the literature: if the sorting bottleneck is indeed a real barrier, the Radix Heap should outperform the binary heap approach, especially on large sparse graphs. Our results confirmed this expectation – as graph size grew, the Radix Heap algorithm consistently ran faster, mirroring the theoretical improvements in the 2025 study. In summary, the experiment demonstrated the key insight from over half a century of research: by adopting new techniques to relax the sorting requirement (as foreshadowed by Bellman’s dynamic programming relaxation and realized in the 2025 algorithm), we can achieve tangible performance gains over classical Dijkstra’s method. This hands-on evaluation thus serves as a practical validation of the literature’s progression, from Bellman and Dijkstra’s foundational ideas to state-of-the-art advancements in shortest path algorithms.

III. EXPERIMENTATION AND PERFORMANCE EVALUATION

This section presents the methodology and evaluation results used to compare the classical Dijkstra algorithm with the Breaking Sorting Barrier SSSP algorithm. Both algorithms were implemented in C++ and executed using the same benchmarking framework to ensure fairness, reproducibility, and consistent measurement across multiple graph sizes.

A. Implementation Details

All experiments were conducted using a custom C++ benchmarking program. Graphs were represented as adjacency lists, where each node stores a vector of outgoing edges. Each edge is defined by a destination node and a 64-bit unsigned integer weight:

```
struct Edge { int to; uint64_t weight; };
```

Graphs were loaded from text files using the `read_graph_from_file` function, which parses input lines of the form:

```
<from> <to> <weight>
```

Lines beginning with ‘#’ or empty lines were ignored. All nodes were assumed to be zero-indexed. Since the parser scans

the entire file to determine the maximum node identifier, the number of nodes is inferred directly from the dataset.

B. Algorithms Tested

Two algorithms were evaluated:

- **Dijkstra (Binary Heap):** implemented with `std::priority_queue` using a min-heap ordering. This represents the conventional $O(E \log V)$ implementation.
- **Breaking Sorting Barrier SSSP:** implemented using a 64-bit **Radix Heap**, a bucket-based priority structure optimized for monotonic keys. This achieves the improved performance characteristics described in recent SSSP research.

The Radix Heap uses 65 buckets to support 64-bit integer distances, and it maintains a monotonic “last extracted” value to ensure correctness.

C. Benchmarking Procedure

Performance was measured using the `time_algorithm` function, which executes the selected SSSP algorithm repeatedly and records the execution time using `std::chrono::steady_clock`. Each experiment was repeated **1000 runs**, and the following metrics were collected:

- **Average runtime (ms)** over all runs
- **Best runtime (ms)** across the runs
- **Distance vector** for verification

Only the distances from the **first run** were stored for correctness checking; subsequent runs were used solely for timing stability.

D. Correctness Verification

After both algorithms completed execution, their output distance vectors were compared using the `verify_results` function. If any discrepancy was found at any node index, execution was terminated with an error message indicating the mismatch.

Across all datasets tested, the distances produced by the Radix Heap SSSP implementation matched those generated by Dijkstra’s algorithm exactly, demonstrating correctness.

E. Datasets

Four graph datasets of increasing size were used:

- `small_graph.txt`: 100 nodes
- `medium_graph.txt`: 1,000 nodes
- `large_graph.txt`: 50,000 nodes
- `extreme_graph.txt`: 1,000,000 nodes

Each dataset consisted of directed weighted edges, and all weights were non-negative 64-bit integers, in accordance with the algorithm requirements.

F. Runtime Results

The experiments show that the Breaking Sorting Barrier SSSP algorithm consistently outperforms the classical Dijkstra implementation, with the performance gap widening as the graph size increases.

For the 50,000-node dataset, Dijkstra required approximately 11.655 ms on average, while the Radix Heap SSSP completed in 3.693 ms, achieving roughly a $3\times$ speedup.

For the 1,000,000-node dataset, Dijkstra’s average runtime rose to 906.330 ms, whereas the Breaking Sorting Barrier SSSP completed in 261.627 ms, giving a speedup of approximately $3.4\times$.

Both algorithms maintained stable runtimes across 1000 runs, and best-run performance followed the same pattern as the averages.

G. Summary

The experiment demonstrates that:

- The Radix Heap SSSP algorithm is fully correct, matching Dijkstra’s output on all datasets.
- The algorithm achieves substantial and increasing speedups as graph size grows.
- The benchmarking methodology provides stable and reproducible results due to repeated-run averaging.

These findings confirm that the Breaking Sorting Barrier SSSP algorithm is a practical and efficient alternative to the classical Dijkstra implementation for large-scale graph processing.

IV. CONCLUSION AND FUTURE WORK

Left blank intentionally

REFERENCES

- [1] R. Bellman, “On a routing problem,” *Quarterly of Applied Mathematics*, vol. 16, no. 1, 1958.
- [2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, 1959.
- [3] C. C. Aggarwal, “A survey on graph algorithms for real-world networks,” in *Managing and Mining Graph Data*. Springer, 2016.
- [4] R. Duan, J. Mao, X. Mao, X. Shu, and L. Yin, “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths,” *Proceedings of the ACM Symposium on Theory of Computing (STOC)*, 2025.